

National College of Ireland

HDWD_SEP24OL

2024/2025

Stephen Hughes

24158071

x24158071@student.ncirl.ie

Filí App - An Irish Language Rhyming Tool

Technical Report



National
College *of*
Ireland

National College of Ireland
Project Submission Sheet – 2024/2025

Student Name: Stephen Hughes

Student ID: x24158071

Programme: HDWD_SEP24OL

Year: 2024/2025 September intake

Module: Project (HDSDEV_SEP24)

Lecturer: Hamilton Niculescu

Submission Due Date: 09/08/25

Project Title: Fill App - An Irish Language Rhyming Tool

Word Count (excluding bibliography and appendices): 8005

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Signature: Stephen Hughes

Date: 09/08/25

Table of Contents

Table of Figures	4
Glossary, Acronyms, Abbreviations and Definitions	5
Executive Summary	7
1. Introduction	8
1.1. Background	9
1.2. Aims	11
1.3. Technologies	12
2. System	15
2.1. Requirements.....	15
2.1.1. Functional requirements	15
2.1.2. Data requirements	17
2.1.3. User requirements	18
2.1.4. Environmental requirements	20
2.1.5. Usability requirements	22
2.2. Design and Architecture	24
2.3. Implementation	35
2.4. Testing.....	42
2.5. Graphical User Interface (GUI) Layout	43
3. Conclusions.....	46
4. Further development or research.....	47
5. References.....	48
6. Appendix	50
6.1. Project Proposal.....	50
6.2. Project Plan.....	50
6.3. Requirement Specification	50

Table of Figures

A table containing all the figure number, description and page

Number	Description	Page
1	Figure 1: ER Diagram of DB Scheme	18
2	Figure 2. System Architecture Diagram	26
3	Figure 3. Rhyming Dictionary Sequence Diagram	29
4	Figure 4. Spelling Game Sequence Diagram	31
5	Figure 5. Multiple Choice Game Sequence Diagram	33
6	Figure 6. Authentication Flow	34
7	Figure 7. Rhyme Cache Code Snippet from rhymeCache.js	36
8	Figure 8. Normalisation Function from rhymeRoutes.js	37
9	Figure 9. Fuzzy Matching Code Snippet from rhymeRoutes.js	38
10	Figure 9. Fuzzy Matching Code Snippet from rhymeRoutes.js	39
11	Figure 10. Word Matching Code Snippet from gameRoutes.js	40
12	Figure 10. Word Matching Code Snippet from gameRoutes.js	41
13	Figure 12. Code snippet from the authRoutes.js file showing Signup and Login routes.	42
14	Figure 13. Section from the formRoutes.js file showing owner-only update route for posts	43
15	Figure 14. Landing Page GUI	46
16	Figure 15. Dashboard Page GUI	46
17	Figure 16. Mobile Screens GUI	47

Glossary, Acronyms, Abbreviations and Definitions

Foclóir Rímeanna 1.0: A document containing all the rhymes used in this project, sourced from <https://cadhan.com>.

UI (User Interface): The visual and interactive elements that the user engages with.

UI (User Interface): The visual and interactive elements that the user engages with.

UX (User Experience): The overall experience and satisfaction a user has when interacting with the application.

SQL (Structured Query Language): Language used for managing and querying relational databases.

API (Application Programming Interface): A set of functions and protocols enabling communication between software components.

DB (Database): Structured collection of data stored and managed electronically.

SPA: Single Page Application. Web App that loads only one page and the dynamically updates components in that page.

MVP (Minimum Viable Product): The simplest version of a product that is usable and functional for release.

Fada: Acute accent in Irish (´) placed over vowels (á, é, í, ó, ú).

Diacritics: Marks added to letters to indicate pronunciation changes.

Normalisation: Process of removing diacritics for matching purposes.

Fuzzball.js: JavaScript library for fuzzy string matching.

Axios: JavaScript library for handling HTTP requests.

Teanglann: Official online Irish dictionary and grammar resource from Foras na Gaeilge (teanglann.ie).

Tearma: Online Irish phrase and terminology resource from Foras na Gaeilge. (tearma.ie)

Foclóir: Online English to Irish dictionary from Foras na Gaeilge. (focloir.ie)

An Foclóirí: a Teanglann plugin that turns words on a page into links, opening a small side window with the dictionary entry so the user is not interrupted. It requires jQuery; in this instance the React jQuery plugin is used, include the script, and call `cab.clickify()`, optionally on a specific container.

Phonetic Description: Written representation of pronunciation.

Executive Summary

This report documents the research, design, implementation, and evaluation of a web-based Irish-language rhyming tool. The platform's primary audience includes language learners, poets, songwriters, and educators seeking to expand their Irish vocabulary in a creative and engaging way.

The core features include:

- **Rhyming Dictionary:** User searches for a list of words that rhyme with their inputted word.
- **Rhyme Game (Spelling):** User guesses a rhyming word with strict spelling and fada accuracy.
- **Multiple Choice Rhyme Game:** Select the correct rhyme from four options.
- **Community Forum** – Post and comment on Irish-language content.

The application uses React.js for the front end, Node.js for backend API logic, and SQL Server for relational data storage (accessed via Docker and Azure Data Studio). Authentication uses Passport.js with role support for user, mentor, and admin (though only user-level access is fully implemented).

The project integrates fuzzy string matching via Fuzzball.js, a Teanglann.ie dictionary plugin via jQuery (react-jquery-plugin), and includes responsive styling through Bootstrap. The backend is optimised by preloading rhyme data into a NodeCache instance to avoid repeated database calls during fuzzy searches.

Despite time constraints and technical challenges, the MVP delivers most primary functionality with the potential for future expansion.

1. Introduction

This document outlines the functional and non-functional requirements for the development of an Irish-language rhyming dictionary web application. The project combines educational, linguistic, and community-building goals to create a platform that encourages users to interact with Irish through rhyme, poetry, and song.

The app's concept emerged from a recognised gap in the availability of modern, creative Irish-learning tools, particularly those that combine phonetic awareness with creative writing. Surveys from both StudyClicks and Gaelchultúr have shown strong dissatisfaction with the way Irish is taught, with a majority of learners expressing a desire to improve their fluency but lacking resources that make the process enjoyable and personally relevant.

The system will cater to multiple user groups, including:

- Language learners seeking vocabulary expansion through rhyme and repetition.
- Poets and songwriters wishing to find inspiration and gain accuracy in rhyme whilst building vocabulary.
- Educators who may use the platform to encourage creative exercises in class.
- Casual users with an interest in exploring the sounds and patterns of Irish words.

The application supports both individual learning and community interaction. Individual learning is promoted through the Rhyming Dictionary and the Rhyme Games (Spelling and Multiple Choice), which provide immediate feedback on spelling, accent placement, and rhyme accuracy. Community interaction is encouraged through the Forum feature, where users can share creative works, comment on others' posts, and suggest missing rhymes to help grow the database.

Technologically, the system integrates a SQL Server database containing over 200 pages of rhyme group data, a Node.js backend with RESTful APIs, and a React.js front end for dynamic, responsive interfaces. The design prioritises accessibility, responsiveness, and linguistic accuracy, incorporating features such as:

- Exact, normalised, and fuzzy matching to handle variations in spelling and fada placement.
- NodeCache preloading to improve performance during fuzzy searches.
- Teanglann.ie integration for reference to definitions and proper spelling.

In its current form, the app serves as a Minimum Viable Product (MVP) that meets the essential requirements for functionality and user experience, while providing a foundation for further development in areas such as curriculum alignment, advanced gamification, and pronunciation support.

1.1. Background

The Irish or Gaeilge, is considered an endangered language by the EU and faces ongoing challenges in terms of how it is taught, learned, and used in daily life. A survey conducted by StudyClicks with over 1,300 secondary school students revealed that 81% believe the way the language is currently taught is outdated (BreakingNews.ie, 2025). This notion is mirrored in findings from a Gaelchultúr and Údarás na Gaeltachta survey of 1,000 participants, which reported that 66% regret not having better Irish, and 74% have the desire to improve their skills (Údarás na Gaeltachta, 2024).

These figures highlight a clear need for innovative, engaging learning resources that go beyond traditional methods. The research underpinning this project indicates that creative writing and rhyme-based exercises can offer such innovation. A study by Yasemin Kirkgoz demonstrates the value of creative writing in language learning, noting that significant progress can be achieved

when learners are personally engaged with thematic material. Citing Lazar (1996, p. 773), Kirkgoz points out that when learners engage emotionally, such as through poetry, they “begin to own the language they learn more fully.”

The question of whether gamification can enhance language learning is more complex. Research from the School of Foreign Languages and Literatures at Chongqing Normal University suggests that emotional engagement with gamified tools was moderate at 48% (Luo, 2023). However, these figures can vary widely depending on the user group. In a rhyme-focused app, for example, poets, songwriters, and creatively inclined learners are likely to engage more deeply than those with only academic goals.

Although there is little research specifically on rhyme-based gamification for language learning, existing studies strongly support the role of rhyming in developing phonological awareness. The comprehensive research of Soňa Grofčíková and Monika Máčajová (2021) found that nursery rhyme learning helped improve children’s phonological awareness which is a foundation for reading and writing skills.

It is important here to distinguish phonological awareness from phonemic awareness:

Phonology is “the study of speech sounds of a language or languages, and the laws governing them,” focusing on the patterns and rules for combining these sounds (ScienceDirect, 2016).

A phoneme is the smallest unit of speech that can distinguish one word from another, such as the /p/ sound in “tap” versus “tab” (Britannica, 2025).

The Grofčíková and Máčajová study also found that while rhyme production scores were generally low (which is likely due to the cognitive effort involved), these scores improved with age. More importantly, once a rhyme association had been learned, vocabulary retention rates remained high across all age groups.

From this, two conclusions emerge:

- Rhyming does not directly create phonemic awareness, but it strongly supports phonological and phonetic skill development.
- Rhyme enhances memory retention, especially when linked to personal creativity and engagement.

These findings form the basis for the decision to build an Irish-language rhyming dictionary with gamified features and community tools. The goal is to combine structured learning with creative play, offering both practical language support and a platform for artistic expression.

1.2. Aims

The primary aim of this project is to design, develop, and deliver a web-based rhyming dictionary for the Irish language that combines linguistic accuracy, gamified learning, and community interaction. The system should:

- Enable users to search for rhyming words from a comprehensive database using exact match, normalised match (ignoring fadas), and fuzzy matching functions.
- Provide interactive rhyme-based games to reinforce correct spelling, pronunciation, and accent placement.
- Facilitate a community forum where users can share creative works and exchange feedback.
- Support future scalability to include pronunciation audio, images, and curriculum-aligned learning levels.

1.2.1. Aims Not Achieved in This Release

- Allow users to submit missing rhymes for admin approval, helping to expand the database organically.
- Allow users with a dedicated Admin role to update the database based on the form above.

1.3. Technologies

This project was built using a full-stack web development approach where each technology was chosen to meet a specific functional or performance need.

React.js: Used to create the interactive front-end of the application. React's component based approach allows for reusability across different pages (e.g., dashboard layout, dictionary search, forum posts) and ensures that only the parts of the interface that need to update will change in real time, improving performance and user experience.

React Router DOM: Manages client-side navigation between different views of the app without requiring a page reload. It was also used to implement protected routes, ensuring that only authenticated users can access certain components like the games and forum.

Node.js with Express.js: Forms the backbone of the server-side logic. Node.js handles requests from the front end, processes them, and interacts with the database before sending the appropriate response. Express.js simplifies route creation and middleware handling, making it easier to structure the backend so that features like authentication, rhyme searches, and forum posting are kept separate and manageable.

SQL Server (via Docker + Azure Data Studio): The project runs SQL Server in Docker so the database setup is the same on any computer, without a heavy local install. It can be started or stopped with a single command, data can be kept between runs or reset quickly for a clean slate, and everything stays isolated from the host machine. Azure Data Studio adds a clear, friendly GUI to run queries, check tables, and export data. This setup matches the databases module I used in the course and this familiarity was helpful.

mssql: The Node.js driver for SQL Server, enabling secure, parameterised queries. This prevents SQL injection attacks and ensures reliable communication between the backend and the database.

Passport.js: Handles user authentication and session management. It ensures that only registered and logged-in users can access restricted features like posting in the forum or submitting rhyme suggestions. While roles for user, mentor, and admin were planned, only the standard user authentication was fully implemented in this version.

bcryptjs: Used for hashing passwords before storing them in the database, ensuring that sensitive user data is never stored in plain text.

express-session: Maintains server-side sessions, allowing logged-in users to remain authenticated across multiple requests and page loads.

connect-flash: Displays short-lived feedback messages during actions like login failures or form errors, improving the overall user experience.

dotenv: Loads environment variables from a .env file, keeping sensitive values (like database credentials and session secrets) out of the public source code.

Fuzzball.js: A JavaScript library that enables fuzzy string matching. This was integrated into the rhyming dictionary to handle near-miss spellings and variations in fada placement, helping users still get results even if their spelling is not perfect.

NodeCache: Improves performance when running fuzzy matching by storing preloaded rhyme data in memory. This avoids repeatedly querying the database for the same set of words during searches, which speeds up response times.

Bootstrap: Used to make the site responsive and mobile-friendly without having to write all CSS from scratch. The grid system and prebuilt components ensured consistent styling across features like forms, navigation bars, and content cards.

Axios: Handles communication between all client-server calls in this app. React talks to the Express API; the server then serves dictionary and game results from the in-memory RhymeCache, and uses SQL Server for auth and forum writes. One shared Axios setup adds tokens, timeouts, and consistent JSON/error handling.

jQuery + Teanglann.ie Plugin: jQuery was used solely for integrating the Teanglann.ie foclóir feature, where each word is turned into a clickable link that opens a pop-up window with a dictionary definition.

The dictionary widgets were embedded into the Dashboard page of the app using iframes. These widgets provide users with external references for spelling and definitions, adding a useful additional layer without requiring the app to host this dictionary data directly.

Tools were picked for how they fit together. React handles the interface, Node/Express runs the logic and security, and SQL Server stores the data. Bootstrap, Axios, and an in-memory cache keep pages responsive, requests consistent, and lookups fast throughout development.

2. System

The system is a web-based platform, and users interact with the application through a responsive browser interface that adapts to desktop, tablet, and mobile devices. Access to most features requires account creation and login, with authentication managed by Passport.js and sessions stored server-side via express-session.

2.1. Requirements

The initial requirements were documented in the original Requirements Specification. While most core features were implemented as planned, some requirements evolved during development due to technical constraints, performance considerations, or scope adjustments.

2.1.1. Functional requirements

The functional requirements were originally defined in the Requirements Specification. Below, they are separated into achieved and unachieved features, reflecting the final implementation status at the end of the project. The requirements have changed slightly to prioritise the spelling and multiple choice game features over the unachieved requirements.

Achieved Requirements:

R1: User Sign-Up: Users can create an account by entering a username, email, and password. Data is validated, hashed using bcryptjs, and stored in the SQL Server database. (Implemented as planned)

R2: User Authentication: Registered users can log in with their email and password. Session persistence is managed with express-session. (Implemented as planned; role-based access structure exists but is not yet fully utilised)

R3: Access Rhyming Dictionary: Logged-in users can search for a word and receive rhyming results based on exact match, normalised match, or

fuzzy match. Fuzzy matching is powered by Fuzzball.js and optimised using NodeCache for faster results. (Implemented as planned)

R4: CRUD Forum Posts: Users can create, view, edit, and delete forum posts. (Implemented with basic CRUD functionality; no ownership validation yet)

R5: Comment on Forum Posts: Users can create, edit, and delete comments on posts. (Implemented with same limitation on ownership checks as forum posts)

R6: Spelling Game: Users are shown a base word and must enter a rhyming word with exact spelling and fada placement. Validation enforces accuracy, and fuzzy logic is intentionally excluded to encourage correct spelling. (Implemented as designed)

R7: Multiple Choice Game: Users see a base word and choose the correct rhyme from four options. One option comes from the same rhyme group and three are distractors from other groups; the answer is checked against the base.

Unachieved Requirements:

R8: Submit Missing Rhyme Form: Intended to allow users to suggest missing words or rhymes for admin review. The form and submission process were not implemented in the final build.

R9: DB CRUD Updates (Admin) : Admin interface for reviewing and approving or rejecting user-submitted rhymes was not implemented due to time constraints. The suggestion data is stored and viewable in the database but cannot yet be actioned via a front-end admin tool.

2.1.2. Data requirements

The rhyming data for this project come from Caoimhín Ó Scanail's Foclóir Rímeanna 1.0 published in 2019; the PDF list was extracted, converted to CSV, and imported into SQL Server for use in the application (Ó Scanail, 2019).

The core data model links five entities so features behave predictably. Each word stores a canonical spelling with fadas, a normalised spelling without diacritics, and a required group_id. That group_id is a foreign key to RhymeGroups, so one rhyme group relates to many words and a single match can expand to a full rhyme set. Forum posts belong to exactly one user through user_id to users.id, and forum comments belong to exactly one post and one user through post_id to forum_posts.id and user_id to users.id. Users enforce unique email and unique username and store a password hash, role, and created_at for provenance. To support fast lookups the words table must allow indexed access by word, by word_normalised, and by group_id, and the application mirrors this with an in memory cache for read paths. These relationships and constraints ensure accurate rhyme retrieval, secure and attributable community content, and consistent joins across dictionary, games, and forum.

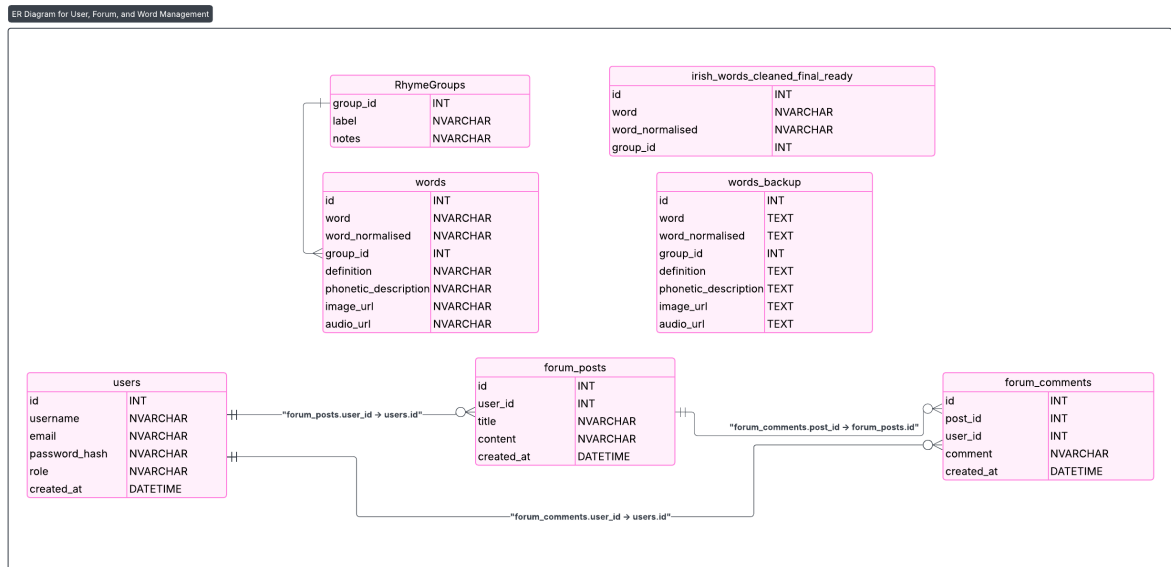


Figure 1: ER Diagram of DB Scheme

The data requirements ensure that:

1. All dictionary and rhyme-related information is structured for quick retrieval and accurate matching.
2. User data is handled securely with encrypted passwords and role storage for future functionality.
3. Forum content is relationally linked to its authors to maintain context and allow potential future ownership checks.
4. Performance is optimised through the strategic use of caching for high-frequency lookups.

This combination of structured relational storage and performance-focused caching supports the application's dual goals: delivering accurate linguistic results and providing a smooth, responsive user experience.

2.1.3. User requirements

The user requirements outline what the intended audience of the Irish-language rhyming dictionary application needs in order to achieve their goals effectively. These requirements focus on usability, accessibility, and feature availability, ensuring that the system meets the needs of different user groups from language learners to creative writers.

The system was designed with three planned user roles: user, mentor, and admin. While role-based permissions were not fully implemented in this release, the core user experience requirements were still met for the standard user role.

The main user requirements include:

Account Management

- Ability to sign up with a username, email, and password.
- Ability to log in securely and maintain a session while navigating the app.

- Passwords stored securely using hashing (bcrypt).

Rhyming Dictionary Access

- Ability to search for an Irish word and retrieve rhymes via:
 1. Exact match (with fadas).
 2. Normalised match (fadas removed).
 3. Fuzzy match (using Fuzzball.js).
- Search results displayed clearly, with clickable words linking to Teanglann.ie dictionary definitions.

Community Interaction

- Ability to create forum posts to discuss Irish language topics, creative writing, or song lyrics.
- Ability to comment on forum posts to engage in discussions.
- View posts and comments in reverse chronological order.

Game Features

- Ability to play a spelling-based rhyme game where correct spelling and fada placement are required.
- Ability to play a multiple-choice rhyme game with two attempts per question.

Learning Support

- Access to embedded dictionary widgets from Teanglann.ie for quick word definition lookups.
- Simple, mobile-friendly navigation using Bootstrap to ensure accessibility on different devices.

In summary, the user requirements emphasised:

Ease of use: users should be able to sign up, log in, and begin using the core features without a steep learning curve.

Linguistic accuracy: rhyme searches should return reliable results, even if spelling variations occur.

Community building: the forum provides a shared space for discussion, encouraging both creative and educational engagement.

Motivation through gameplay: game modes provide alternative learning methods to keep users engaged.

These requirements ensured that the application was not just a tool for rhyme lookups, but also a platform for community learning and creative collaboration.

2.1.4. Environmental requirements

The environmental requirements outline the hardware, software, and platform conditions necessary for developing, running, and maintaining the Irish-language rhyming dictionary application. These requirements are divided into development, deployment, and target user environments.

The project was built using a full-stack JavaScript setup with a React.js front end, Node.js/Express.js backend, and SQL Server database. SQL Server was run inside a Docker container and managed using Azure Data Studio. This setup was chosen not only for consistency between development and deployment but also to provide additional hands-on practice in preparation for a repeat assessment in databases.

Development Environment Requirements

Hardware:

- MacBook Pro (2015) with 16 GB RAM.
- At least 10 GB free storage for Node modules, Docker images, and database files.

Software:

- Node.js (v20+) and npm for running and managing project dependencies.
- Docker Desktop to host the SQL Server instance.
- Azure Data Studio for database management, running queries, and checking table structures.
- Visual Studio Code as the main code editor.

Deployment Environment Requirements

Backend:

- Node.js runtime environment (v20+).
- SQL Server instance accessible via Docker or remote host.
- Sufficient server memory to allow preloading of rhyme data into NodeCache for faster search performance.

Frontend:

- Modern web browser capable of handling React applications.
- Internet access to fetch data from the backend and load Teanglann.ie dictionary widgets.

Target User Environment

Devices: Desktop, laptop, tablet, and mobile devices.

Browsers: Chrome, Firefox, Safari, Edge (latest versions).

Screen Resolution: Fully responsive design supporting widths down to 360px (mobile portrait view).

Connectivity: Recommended minimum of 1 Mbps for smooth operation.

In summary, the chosen environment balances ease of setup and practical performance needs. Running SQL Server via Docker with Azure Data Studio allowed for a portable, consistent database environment while reinforcing database management skills in a realistic development workflow.

2.1.5. Usability requirements

The usability requirements define how the system supports learnability, efficiency, and error tolerance for Irish-language learners and creative users. Given the project's goals, the interface emphasises clear navigation, fast feedback, and language-specific aids (e.g., fada handling and dictionary references). The design uses Bootstrap for responsive layouts, React Router DOM for predictable navigation, and consistent UI patterns across the dictionary, games, and forum.

Achieved Usability

Clear, consistent navigation

- A simple, predictable layout with a left or top navigation (depending on viewport) that groups features: Rhyming Dictionary, Games, Forum, Dashboard.
- Protected routes ensure only logged-in users see interactive features, reducing confusion.

Responsive, mobile-first design

- Bootstrap grid/components ensure the app works on mobile, tablet, and desktop.
- Key actions (Search, Submit, Post and Play: “Try Another”, “Check”, “Next Question”) remain visible and finger-reachable on small screens.

Fast feedback and clear states

- Dictionary search returns results quickly (under normal conditions).
- Obvious empty states (“No rhymes found: check spelling or try another word”).
- Success/failure messages on login, posting, and game answers.

Language-aware input handling

- Rhyming Dictionary accepts both exact spellings (with fadas) and normalised input (without fadas), plus fuzzy match for near-misses.
- Spelling game intentionally requires exact spelling (including fadas) and explains why an answer is rejected (misspelling vs. wrong rhyme group).

Embedded learning support

- Teanglann.ie widgets are available where helpful, so users can check definitions/spellings without leaving the app. Dictionary, translation and terminology search boxes on the dashboard and an Focloirí on the rhyming dictionary component.

Form clarity and validation

- Required fields marked; basic client/server validation prevents empty submissions.
- Button labels are action-oriented (“Search Rhymes”, “Check Answer”, “Post”).

Readable typography and contrast

- Bootstrap defaults and spacing rules give consistent line length, padding, and contrast for most users.

Keyboard-friendly basics

- Forms and buttons are reachable with Tab/Enter; no critical action is mouse-only.

Accessibility depth

- Very little has been done for accessibility other than the inclusion of alt text on images.

Error messaging specificity

- Some errors are generic (e.g., login failure). Clearer, field-level messages and inline hints would reduce user effort.

2.2. Design and Architecture

This section outlines how the system is put together and how the main algorithms work. The design is layered and cache first. On startup the server loads the dictionary into memory, and all rhyme lookups and game actions read from that cache. The database is the source of truth for saving data and for the initial load, but it is not used on the hot path, which keeps responses fast and predictable.

At a high level the client is a React single page application, the middle tier is an Express and Node API, and storage is SQL Server running in Docker and managed with Azure Data Studio. The API is kept small and grouped by feature, covering authentication, the rhyming dictionary, the games, and the forum. Controllers stay thin and hand work to focused services, while shared concerns such as sessions and caching are handled in one place. An architecture diagram can be included to show these pieces and their data flow.

Architectural overview

The client is a React single page app. It shows the dictionary, spelling game, multiple choice game, forum, and sign in screens, and talks to the server with Axios. React Router handles navigation so pages do not reload. A small jQuery hook opens a Teanglann window so learners can view definitions without leaving the page.

- AuthService integrates Passport and bcrypt with express session to create and validate sessions.
- RhymeRoutes implements the rhyme lookup pipeline, moving from exact to normalised to fuzzy, and reads from the cache first.
- GameRoutes implements both games using only the cache. The spelling game requires exact spelling with correct fadas. The multiple choice game builds one correct option from the same rhyme group and three from other groups.
- ForumRoutes provides simple create and read for posts and comments.

A small repository wraps the mssql driver with clear, parameterised queries. It warms the cache at startup, checks users during auth, and saves or reads forum content. The database is SQL Server in Docker. Core tables are words with word, word normalised, and group id, rhyme groups, users, forum posts, and forum comments. At startup the server reads id, word, word normalised, and group id once and builds the in memory indices so dictionary lookups and game checks stay fast.

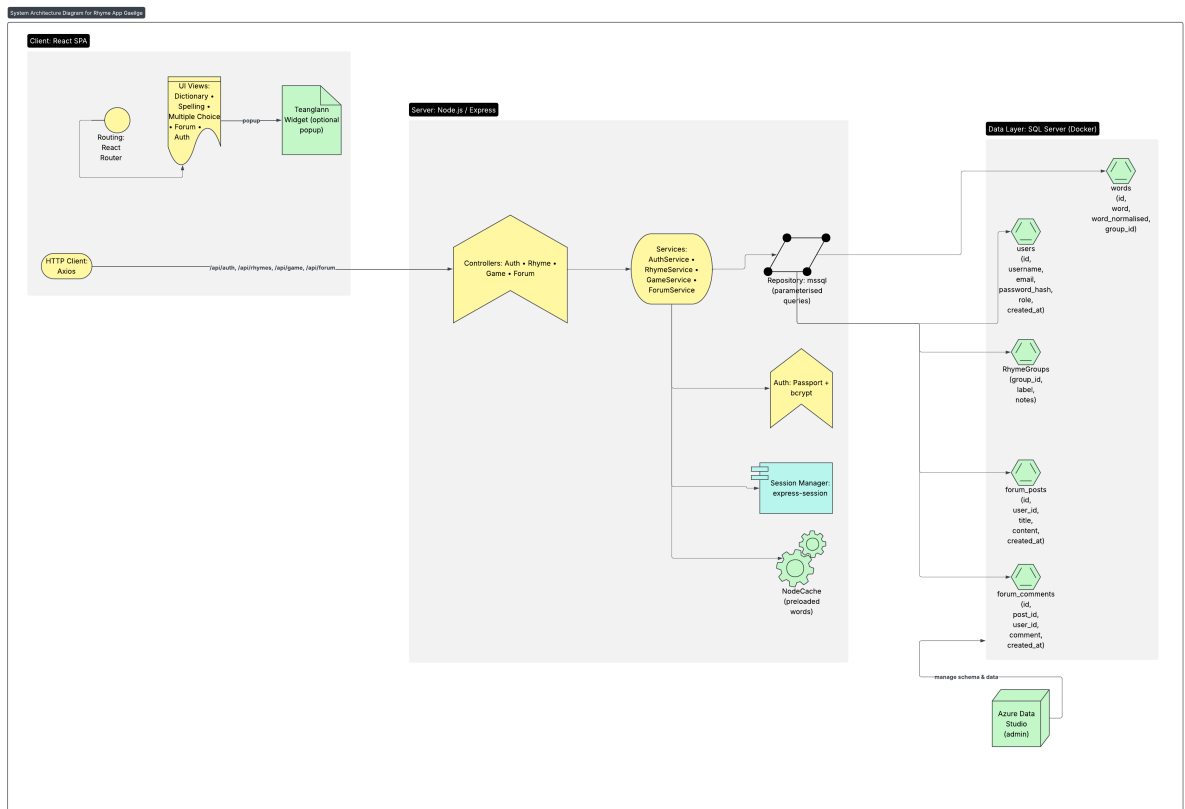


Figure 2. System Architecture Diagram

The diagram above shows a React single page app on the left, an Express API in the middle, and SQL Server on the right. React Router switches screens and Axios sends requests to the API. The API uses small controllers for authentication, the dictionary, the games, and the forum, while focused services handle logic. An in memory cache holds the words table so lookups and game checks are quick, and a simple fuzzy matcher is used only when exact and normalised checks fail. SQL Server running in Docker stores data reliably. This layout is clear, fast, and easy to extend.

Core Algorithms and Component Flows

Cache-first runtime

On startup the server runs a single select on the words table and stores the entire result as one array in memory using NodeCache. Dictionary searches and both games read only from this cached array. The database is used for login, forum reads and writes, and to load or refresh the cache after any admin change. This keeps requests fast and predictable. Exact and normalised checks run quickly, while fuzzy matching scans the cached list and is used only when the first two checks fail.

Fields used during lookups and gameplay

- **word**: Canonical Irish spelling **WITH** fadas. Used for exact matches and for strict checking in the Spelling Game.
- **word_normalised**: Diacritics/ fadas removed and lowercased. Used for the second search stage and as the baseline for fuzzy matching.
- **group_id** : Identifier for the rhyme class. Once a match is found, group_id is used to gather the full rhyme set and to validate answers in both games.
- **id** : Stable primary key for each entry. Helpful for deduping, logging, and UI keys, but not essential to the matching logic.

In practice, most queries resolve at the exact or normalised stage. Fuzzy matching is a deliberate last resort that offers a helpful suggestion without adding database load.

Data normalisation and orthography

Irish orthography uses diacritics (fadas) to mark vowel length. The system embraces this reality by making **two parallel representations** of each word:

- The word column stores the **canonical orthography** (with fadas) and is the only reference used by the Spelling Game. When the game says “exact orthography required”, it literally means that the user’s guess must match this word string exactly ie. same letters, same diacritics, no additions or omissions.
- The word_normalised column stores a **diacritic-stripped** and lowercased form. It is used to soften dictionary searches (stage 2) and to prepare a fair input to the fuzzy matcher (stage 3). Programmatically,

normalisation is implemented via Unicode NFD: decompose characters and drop combining marks, then lowercase and trim.

This split allows the dictionary to be forgiving while keeping the Spelling Game strict by design.

Rhyming dictionary algorithm and sequence flow

The lookup runs in three ordered stages that balance accuracy and speed. Stages one and two query the database. Stage three switches to the cached words in memory, which keeps common cases quick and still gives a helpful fallback when spelling is off.

1. **Exact stage.** The server looks up the input exactly as written, including fadas. If a row is found, it reads that row's group id and returns every word with the same group id. The response is marked as an exact match.
2. **Normalised stage.** If no exact hit, the server removes diacritics and lowercases the input, then looks up that normalised value in the word normalised column. If found, it returns the full group and marks the response as a normalised match. This often indicates a missing or misplaced fada.
3. **Fuzzy stage.** If both database checks fail, the server reads the cached words array, compares the normalised input to every normalised word using a Levenshtein based score, and takes the best result. If the best score is sixty or higher, it returns the group for that closest word and includes a suggestion. Very short inputs can be skipped here to avoid noisy scores. If nothing qualifies, the server returns not found. It is used against normalised strings to avoid over penalising missing fadas.



Figure 3. Rhyming Dictionary Sequence Diagram

Technical points

- Fuzzy matching uses Fuzzball's Levenshtein ratio on a scale from zero to one hundred. The acceptance threshold is set to sixty.
- Results include match type, the list of rhymes, and a suggestion only for the fuzzy case

Two practical notes matter here. First, fuzzy matching is deliberately deferred to the end to keep CPU costs low for the common case. Second, very short inputs produce noisy similarity scores; the service may decline to fuzz for inputs below a minimum length to avoid poor suggestions.

Game logic

Both games reuse the same cache and groupings; they differ in how answers are validated.

Spelling game algorithm and sequence flow

The spelling game enforces exact Irish orthography while still giving a gentle first-attempt nudge. All reads come from the cached words array so the flow is fast and predictable.

1. The client asks for a random prompt. The server reads the cached words, counts members per rhyme group, keeps only groups with at least two words, picks one word, and returns it as the prompt.
2. The client posts the user's answer with the prompt word and an attempt counter. The server trims inputs and validates that both fields exist.
3. The server blocks using the prompt itself as an answer by comparing normalised forms. If they are the same it returns incorrect.
4. The server finds the prompt's group id in the cached array. If the prompt is not found it returns 404. It then builds the list of valid rhyme answers for that group including their normalised forms.
5. Exact pass. If the user's input exactly equals any rhyme word including fadas the result is correct.
6. First attempt leniency. If this is attempt one and the normalised input equals any rhyme's normalised form the result is retry so the UI can ask the user to fix fadas.
7. Otherwise the result is incorrect. Server errors return 500.

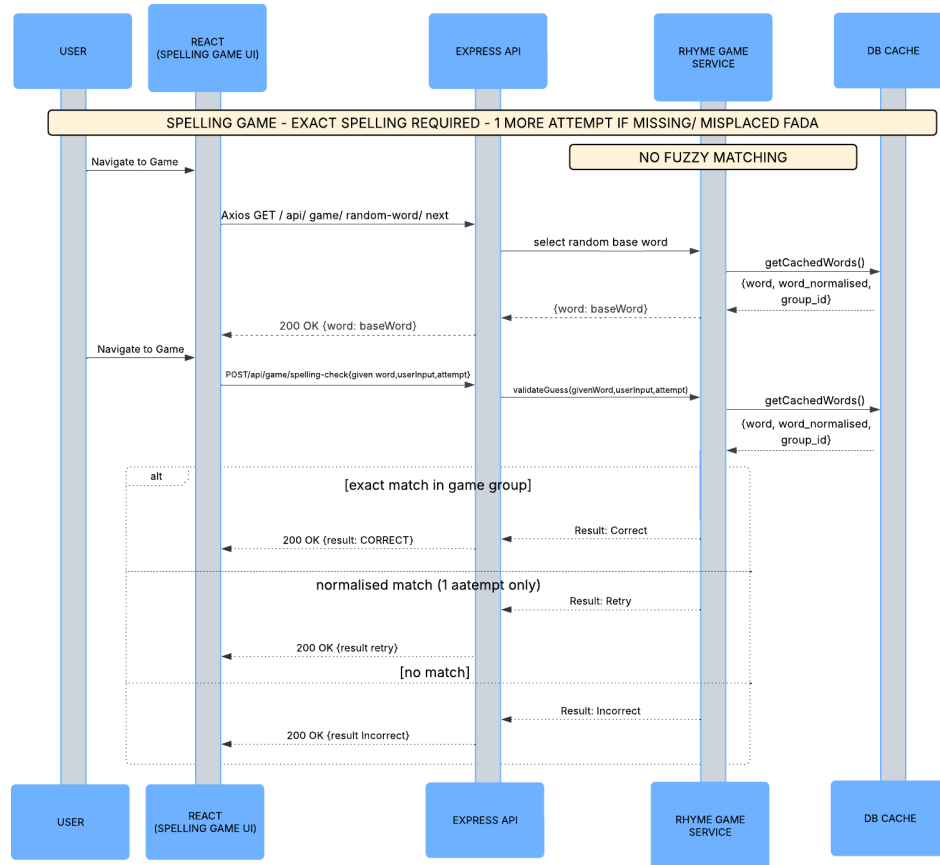


Figure 4. Spelling Game Sequence Diagram

Technical points

- Cache only read path so no database calls while playing
- Exact orthography required for a correct result with fadas enforced
- The prompt word cannot be reused as an answer because normalised forms are compared.
- A single retry hint exists only on the first attempt to encourage proper fadas
- No fuzzy matching is used in this game by design
- Status codes include 400 for missing fields, 404 for an unknown prompt, 503 for an empty cache, and 500 for unexpected errors.

Two practical notes matter here. First, selecting only groups with at least two words prevents dead prompts and keeps rounds meaningful. Second, the first attempt retry helps learners correct fadas without weakening the exact spelling rule.

Multiple choice game algorithm and sequence flow

The multiple choice game checks rhyme knowledge without spelling leniency. All reads come from the cached words array so each round is quick and repeatable.

1. The client requests a new question. The server reads the cached words, picks a random word as the prompt, and notes its group id.
2. The server collects rhyme candidates from the same group but excludes the prompt itself. If none exist it returns not found.
3. The server picks one correct option from those rhyme candidates.
4. The server gathers wrong options by scanning the cached list for words in other groups and excluding the prompt and the chosen correct word until it has three.
5. The server combines the one correct option with the three wrong options to make four choices.
6. The server shuffles the choices using the Fisher Yates method so the correct position varies each time.
7. The server returns the prompt word, the four shuffled choices, and an answer key for the client to check selection. Errors return a server error response.

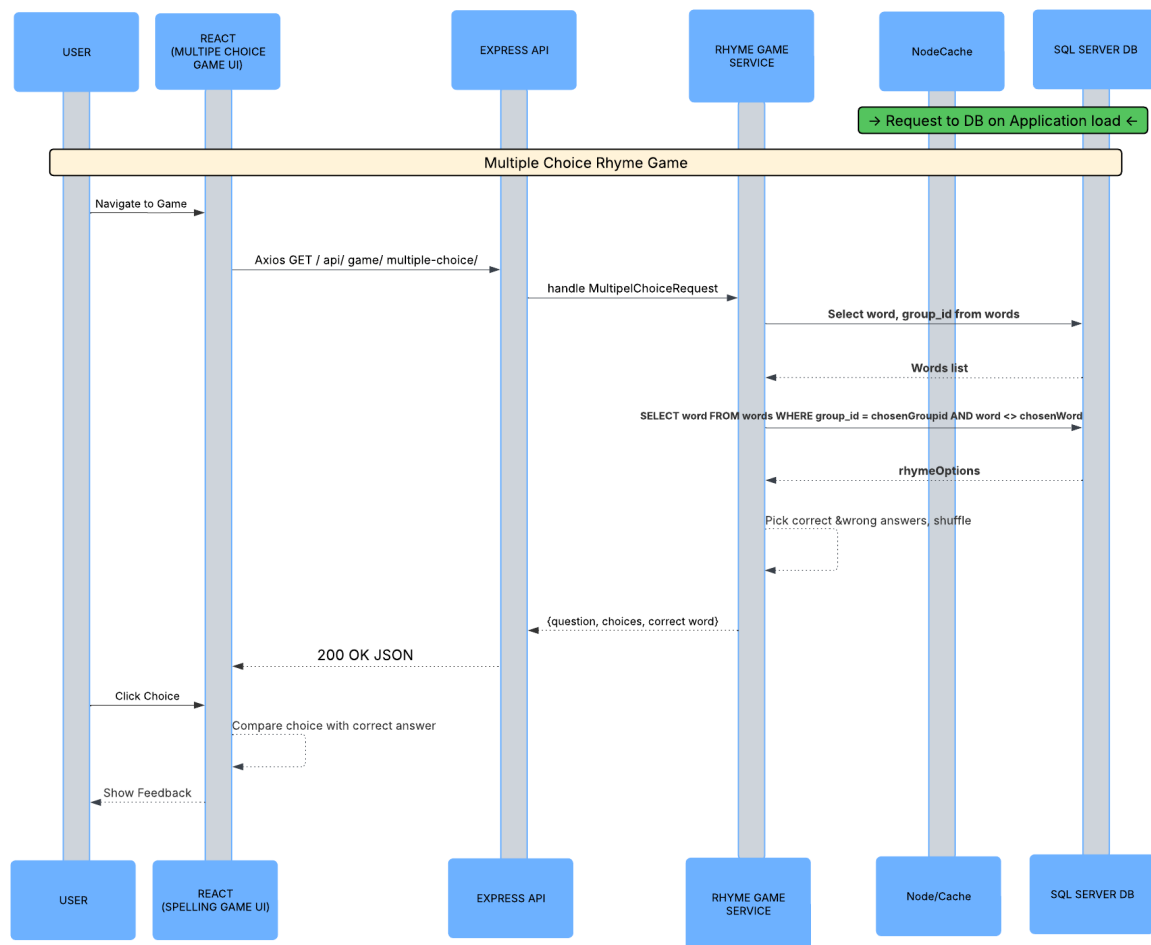


Figure 5. Multiple Choice Game Sequence Diagram

Technical points

- Cache only read path so no database calls while playing
- One correct option always comes from the prompt group
- Three distractors are drawn from other groups to avoid accidental rhymes
- Duplicates are prevented by excluding the prompt and the chosen correct word
- No fuzzy logic is used in this game
- Status codes include 404 when no rhyming candidates are available, 503 when the cache is empty, and 500 for unexpected errors

Two practical notes matter here. First, shuffling prevents users from guessing based on option order and keeps rounds varied. Second, selecting distractors only from other groups maintains question quality by avoiding near rhymes that could confuse learners.

Authentication and Sessions

Authentication uses Passport for credential checking and express-session for session management. On signup, the server hashes the password (bcrypt) and inserts a new row into users, relying on database uniqueness constraints for email and username.

On login, the server retrieves the user by email, compares hashes, and, on success, attaches a session to the client. Logout destroys that session. Protected routes (e.g., posting to the forum or playing games in authenticated mode) are guarded by a simple middleware that checks for the presence of a valid session.

Because authentication is orthogonal to the dictionary and game logic, it appears in those sequence diagrams mainly as a precondition (“user is authenticated”).

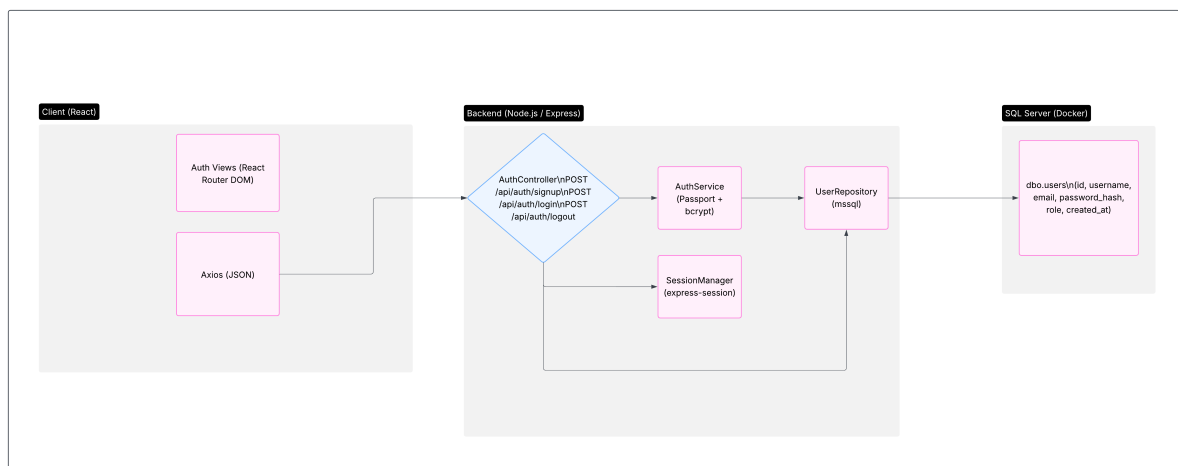


Figure 6. Authentication Flow

Repository and database access

The repository keeps SQL out of controllers and services. Each function is a small, parameterised query, for example getting a user by email, inserting a post, or reading the four columns needed to build the dictionary cache. This explicit approach makes behaviour easy to follow and avoids hidden object tracking. For speed, add three non unique indexes on the dictionary table, one on word, one on word_normalised, and one on group_id. These speed up cache hydration and admin updates without affecting the cache first runtime. The schema is simple. The words table links each entry to a rhyme group through group_id, RhymeGroups stores labels or notes, and users, forum_posts, and forum_comments support the community features. Optional fields such as definition, phonetic description, image url, and audio url are not used in matching but can enrich the UI later.

Error handling, security, and performance

Input validation happens in two places: light checks in the client to guide input, and strict checks on the server to guarantee correctness. Routes return clear JSON with status codes such as 400 for bad input, 401 or 403 for unauthorised access, and 404 for not found. Passwords are hashed with bcrypt and usernames and emails are unique. All SQL uses parameterised queries to prevent injection. Sessions are created on login, regenerated to avoid fixation, and destroyed on logout. From a performance view, the server builds the cache once after startup with a single pass over the dictionary. After that, dictionary and game requests run at memory speed. Fuzzy matching is the only linear step per request and runs only after two faster checks, so it does not dominate typical workloads. If the dataset grows, fuzzy candidates can be bucketed by first letter and length to reduce comparisons without changing behaviour.

2.3. Implementation

This section explains how the system runs in practice, focusing only on the parts that drive the product at runtime: the cache that powers lookups and games, the rhyming lookup pipeline (exact → normalised → fuzzy), the two game flows, authentication with password hashing and sessions, and the forum write path.

Cache as the read model

When the server starts, it connects to SQL Server, runs one query on the words table, and loads the result into memory before serving requests. From then on, the dictionary and both games read from this cache instead of hitting the database on every call. Lookups use three passes: first the exact spelling with fadas, then a normalised form without diacritics, and finally a fuzzy comparison against the list of normalised strings if needed. Because the data is already in memory, responses are fast and predictable. SQL Server still stores everything safely and is used for sign-in, forum posts, and to hydrate or refresh the cache after any change.

```
server > services > JS rhymeCache.js > ...
1 // server/services/rhymeCache.js
2 const { pool, poolConnect } = require('../db');
3 const NodeCache = require('node-cache');
4 const cache = new NodeCache({ stdTTL: 0 });
5
6 async function preloadRhymeData() {
7   await poolConnect;
8   const result = await pool.request()
9     .query("SELECT word, word_normalised, group_id FROM words");
10  cache.set('words', result.recordset);
11  console.log(`Preloaded ${result.recordset.length} rhymes`);
12 }
13
14 function getCachedWords() {
15   return cache.get('words') || [];
16 }
17
18 module.exports = { preloadRhymeData, getCachedWords };
19
```

Figure 7. Rhyme Cache Code Snippet from rhymeCache.js

Diacritic/ Orthography Normalisation

Normalisation is a small utility the server calls before tolerant search and fuzzy. It applies Unicode NFD to split base letters and accents, removes combining marks with a regex, then lowercases and trims. In code it is simply: return

```
word.normalize("NFD").replace(/[  300-  36f]/g, "").toLowerCase().trim();
```

The route uses this to create a no accent version from the user's input and to compare against the word_normalised column. As mentioned previously, the fuzzy step also runs only on these normalised strings and accepts a match at sixty or higher, then maps back to the canonical spelling with fadas for the response. The Spelling Game ignores normalised values and requires the exact canonical form.

```
// Remove accents/fadas from a word so we can compare without diacritics
// Example: "r  " and "ra"
// Source: https://www.codu.co/articles/remove-accents-from-a-javascript-string-skqplnb
function normalise(word) {
  return word.normalize("NFD").replace(/[  300-  36f]/g, '');
}
```

Figure 8. Normalisation Function from rhymeRoutes.js

Rhyming lookup pipeline (dictionary)

The dictionary feature follows a three-stage pipeline, always **cache-first**:

1. **Exact stage.** If the **exact-spelling index** contains the user's query (with fadas), the system reads that entry's rhyme-group id and returns the full set of words for that group from the **group index**. The response is labelled as an exact match.
2. **Normalised stage.** Otherwise, the system computes the normalised form of the query and looks it up in the **normalised index**. If a bucket is found, the first entry provides the rhyme-group id; the **group index** yields the full set of rhymes. The response is labelled as a normalised match.
3. **Fuzzy stage.** If neither stage succeeds, the system compares the query's normalised form against the **candidate list** using a string-similarity score (Levenshtein-ratio family). The best candidate is accepted only if it exceeds a defined threshold. If accepted, the response includes both the rhymes of that candidate's group and a **suggestion** field indicating the closest dictionary word. Otherwise, the request is treated as "no match."

```

// Step 3: Fuzzy match from cache
// If DB searches fail, use the preloaded cache for faster fuzzy matching
const cachedWords = getCacheWords(); // [{word, word_normalised, group_id}, ...]
const wordList = cachedWords.map(w => w.word_normalised); // Compare on normalised form
// fuzzball.extract compares the input against all possible word options, and returns:
// [ [bestMatchString, score], ... ] sorted by score (0-100, higher = more similar).
// Score is built-in (based on edit distance). The top result is taken.
const [bestMatch, score] = fuzzball.extract(noAccent, wordList)[0] || [];

// If the score is 60+ (set threshold), return all rhymes from that match's group
if (score >= 60) {
  let match = cachedWords.find(w => w.word_normalised === bestMatch);
  let rhymes = cachedWords
    .filter(w => w.group_id === match.group_id)
    .map(w => w.word);

  return res.json({
    matchType: 'closest',
    match: match.word,
    rhymes
  });
}

// Error Handling

// Nothing found in any of the 3 steps
res.status(404).json({ message: 'No rhymes found' });

} catch (err) {
  // If any server error happens during the process, log it and send a server error response
  console.error('Error finding rhymes:', err);
  res.status(500).json({ message: 'Server error' });
}
});

```

Figure 9. Fuzzy Matching Code Snippet from *rhymeRoutes.js*

Fuzzy comparison is intentionally last to keep CPU costs low; very short inputs may be excluded from fuzzy matching to avoid noisy suggestions.

Game logic

Both games read only from the in memory cache, so gameplay never touches the database.

Spelling Game (exact orthography)

For each round the server serves a prompt from cache only. It loads the words array, counts members per group, keeps words from groups with at least two members, and returns one at random. If the cache is empty it returns 503, and if no eligible groups exist it returns 404. When the user submits an answer the route trims givenWord and userInput, then blocks reusing the prompt by comparing their normalised forms. It finds the prompt's group_id by exact match on givenWord and builds a list of all words in that same group with both their canonical and normalised spellings.

The first pass checks for an exact match on the canonical spelling including fadas; if found the result is correct. If not, and only on the first attempt, it normalises the user input and compares it to the group's normalised forms; a match here returns retry so the user can fix fadas. Otherwise the result is incorrect. Unexpected errors return 500.

```
// 7) First pass: check for an exact match (includes correct fadas, case, etc.).
for (let i = 0; i < rhymeList.length; i++) {
  if (rhymeList[i].word === userInput) {
    // 8) If exact match is found, the answer is correct immediately.
    return res.json({ result: 'correct' });
  }
}

// 9) If this is the user's first attempt, we allow a "normalised" retry.
//     use strip fadas/accents and compare to stored normalised forms.
if (attempt === 1) { // only offer the retry path on the first attempt
  const userInputNormalised = normalise(userInput);
  for (let i = 0; i < rhymeList.length; i++) {
    // 10) Compare the user's normalised input to the normalised version in the DB.
    //      If it matches a rhyme, we tell the user to allow a retry (to fix fadas).
    if (rhymeList[i].word_normalised === userInputNormalised) {
      // 11) If a normalised match exists on attempt 1, return "retry" so the UI can prompt the user to try again with
      //      proper fadas/spelling.
      return res.json({ result: 'retry' });
    }
  }
}

// 12) If no exact match and (if applicable) no normalised match, it's incorrect.
return res.json({ result: 'incorrect' });
} catch (err) {
  // 13) Any unexpected error returns a 500 so the client can show a generic error message.
  console.error('Error checking spelling:', err);
  return res.status(500).json({ message: 'Server error' });
}
});
```

Figure 10. Word Matching Code Snippet from gameRoutes.js

Multiple-Choice Game

The multiple-choice flow picks a random prompt word, notes its group id, and gathers same group candidates excluding the prompt; if none exist it returns 404.

It selects one correct option from those candidates. It then scans the cached list for words in other groups, skipping the prompt and the chosen correct word, until it has three distractors. The four options are combined and shuffled with the Fisher Yates method, and the server returns the prompt, the shuffled choices, and the correct answer. The client displays the prompt and options, compares the user's click with the returned correct value to show feedback, and then requests a new question.

```
// 2) Pick a random question word from the list
const randomWordIndex = Math.floor(Math.random() * wordsList.length);
const chosenWordRow = wordsList[randomWordIndex];
const chosenWord = chosenWordRow.word;
const chosenGroupId = chosenWordRow.group_id;

// 3) Get words that rhyme with the chosen word (from cache, no DB query)
const rhymeOptions = wordsList.filter(
  w => w.group_id === chosenGroupId && w.word !== chosenWord
);

if (!rhymeOptions || rhymeOptions.length === 0) {
  return res.status(404).json({ message: 'No rhyming words found' });
}

// 4) Pick one correct answer
const correctWordIndex = Math.floor(Math.random() * rhymeOptions.length);
const correctWord = rhymeOptions[correctWordIndex].word;

// 5.) Pick three wrong answers from other groups (from cache)
const wrongWordOptions = [];
for (let i = 0; i < wordsList.length; i++) {
  const candidate = wordsList[i];
  const notSameGroup = candidate.group_id !== chosenGroupId;
  const notChosenWord = candidate.word !== chosenWord;
  const notCorrectWord = candidate.word !== correctWord;

  if (notSameGroup && notChosenWord && notCorrectWord) {
    wrongWordOptions.push(candidate.word);
  }

  if (wrongWordOptions.length === 3) {
    break;
  }
}
}
```

Figure 11. Game Building Code Snippet from gameRoutes.js

Authentication (hashing + sessions)

Authentication uses **password hashing** on signup, **hash comparison** on login, and **server-side sessions** for continuity. The flow is intentionally small and explicit:

- **Signup:** check for an existing account; hash the password; insert the user; attach the new user id to the session.
- **Login:** retrieve user by email; compare the password with the stored hash; attach the user id to the session on success.
- **Logout:** Kill the session.
- **Protected routes:** a simple middleware denies writes if there is no active session id.

```
// Signup route - creates a new user and logs them in immediately

router.post('/signup', async (req, res) => {
  // Get the username, email, and password from the request body
  const { username, email, password } = req.body;

  try {
    // Create the new user in the database (hashes the password before saving)
    const newUser = await createUser(username, email, password);

    // Automatically log in the new user after signup
    req.login(newUser, (err) => {
      if (err) {
        // If there's a problem logging them in, send a 500 error
        return res.status(500).json({ message: 'Login error after signup' });
      }
      // Otherwise, return success message and the new user object
      return res.status(200).json({ message: 'Signup successful', user: newUser });
    });
  } catch (err) {
    // If the error says "Email already", return that message, otherwise say "Unexpected error"
    const msg = err.message.includes("Email already") ? err.message : "Unexpected error";
    res.status(400).json({ message: msg });
  }
});

// Login route - uses Passport LocalStrategy to authenticate the user
router.post('/login', (req, res, next) => {
  // passport.authenticate('local') checks email/username and password using our configured strategy
  passport.authenticate('local', (err, user, info) => {
    if (err) {
      // If something went wrong in the authentication process, send a 500 error
      return res.status(500).json({ message: 'Login error' });
    }

    if (!user) {
      // If no user was found or password was wrong, send a 401 Unauthorized
      // Use the message from Passport's LocalStrategy if possible, otherwise use our own default
      return res.status(401).json({ message: info?.message || 'Invalid credentials' });
    }

    // If the credentials are correct, log in the user and create a session
    req.login(user, (err) => {
      if (err) {
        return res.status(500).json({ message: 'Session error' });
      }
      // Send success message and the logged in user data
      return res.status(200).json({ message: 'Login successful', user });
    });
  })(req, res, next); // This immediately calls the returned function from passport.authenticate
});
```

Figure 12. Code snippet from the `authRoutes.js` file showing Signup and Login routes.

Forum posting (session-gated DB write)

The forum reads and writes directly to SQL Server so new content appears immediately; nothing is cached. The API returns a post list with author info and a live **comment_count** via a join and subquery. Creating, editing, and deleting posts require a logged-in user (`req.user.id`). Missing fields return 400, unauthenticated requests return 401, owner checks are enforced in SQL (**WHERE id = @id AND user_id = @user_id**) so updates and deletes on other people's content return 404, and unexpected errors return 500. Comments follow the same pattern per post. On the client, **Forum.js** loads posts, then fetches comments per post, and provides simple create, edit, and delete flows with success/error messages.

```
// Edit a post (owner only)
router.put('/posts/:postId', async (req, res) => {
  const userId = req.user?.id;
  if (!userId) return res.status(401).json({ message: 'Not logged in' });
  const { postId } = req.params;
  const { title, content } = req.body;
  if (!title || !content) return res.status(400).json({ message: 'Title and content required' });

  try {
    const result = await pool.request()
      .input('id', sql.Int, postId)
      .input('user_id', sql.Int, userId)
      .input('title', sql.NVarChar, title)
      .input('content', sql.NVarChar, content)
      .query('UPDATE forum_posts SET title = @title, content = @content WHERE id = @id AND user_id = @user_id');
    if (!result.rowsAffected[0]) return res.status(404).json({ message: 'Post not found or not yours' });
    res.json({ message: 'Post updated' });
  } catch (err) {
    console.error('Error updating post:', err);
    res.status(500).json({ message: 'Server error' });
  }
});
```

Figure 13. Section from the `formRoutes.js` file showing owner-only update route for posts

Error handling and boundaries

Error handling

Routes return clear JSON with status codes: **400** for invalid input, 401 or **403** for unauthenticated or forbidden actions, **404** when a rhyme or resource is not found, 409 when signup conflicts with an existing email or username, and **500** for unexpected server errors. Inputs are validated on the server, sessions are created and destroyed correctly, and all SQL calls are parameterised to prevent injection.

Boundaries

Boundary rules in this build are simple and consistent. The dictionary and games read only from the in memory cache, while the forum and auth talk directly to SQL Server so new content appears immediately. Fuzzy matching runs only after exact and normalised checks fail, uses a Levenshtein style score, accepts results at sixty or higher, and is never used in the games. The Spelling Game needs an exact same group match with fadas, with a one time normalised retry hint on the first attempt. Multiple Choice picks one correct option from the same group, three distractors from other groups, shuffles them, and checks by group only. Posting or editing requires a logged in session, and updates or deletes succeed only when the record owner matches the session user in SQL. Public endpoints cover auth, dictionary lookups, game rounds, and forum create read update delete, while dictionary edits are not included in this release. Hot path reads use only word, word_normalised, and group_id; optional fields like definitions or media links are stored but not used at runtime. The Teanglann widget opens in a separate window and never redirects the user. Routes return consistent JSON with clear status codes. The database runs in Docker and is managed with Azure Data Studio for a repeatable setup.

2.4. Testing

During exploratory testing I noted three impactful items: first, the Spelling Game previously accepted the base word itself as a correct answer; that's consistent with "same-group" validation, but this has been resolved since.

Second, after changing the navbar to allow layout wrapping, the header behaves correctly across breakpoints and no longer collapses awkwardly on small screens. Third, direct database reads in early game endpoints were both slow and sometimes threw errors; moving the games onto the same cache-backed read model as the rhyming dictionary removed those issues and made responses consistent.

Most important checks and outcomes

- **Authentication:** Signup/login/logout behaved correctly; duplicate email/username is rejected; protected routes return 401 without a session.
- **Rhyming Dictionary:** Exact matches (with fadas) return immediately; normalised queries (without fadas) resolve correctly; fuzzy suggestions seem to appear only above the threshold.
- **Spelling Game:** Enforces exact orthography (fadas required). Using the base word as an answer is currently fixed as to be incorrect.
- **Multiple Choice:** Always one correct option from the base group and three distractors from other groups; options are shuffled; answer checking is by group equality.
- **Forum:** Authenticated users can create posts/comments; unauthenticated attempts are blocked; new content is visible immediately (no cache).
- **Performance & reliability:** Cache warm-up occurs before serving; dictionary and game requests run from memory (no SQL on hot path), fixing the prior latency and query errors.
- **Responsiveness:** Navbar wrap fix resolved container layout issues; pages remained usable on desktop and mobile viewports.

2.5. Graphical User Interface (GUI) Layout

Design influences: Duolingo (clarity, progression), RhymeZone (text-first search), Rhymer's Block (creative/community cues).

Assets: All designed or sourced though Figma when designing wireframes; icons from SVG Repo; background made with Figma's **Grainy Gradient** plugin.

Framework: Bootstrap for grid, spacing, and components; light custom CSS for polish.

Landing page: Simple top navbar + Bootstrap carousel to introduce Dictionary, Games, and Forum.

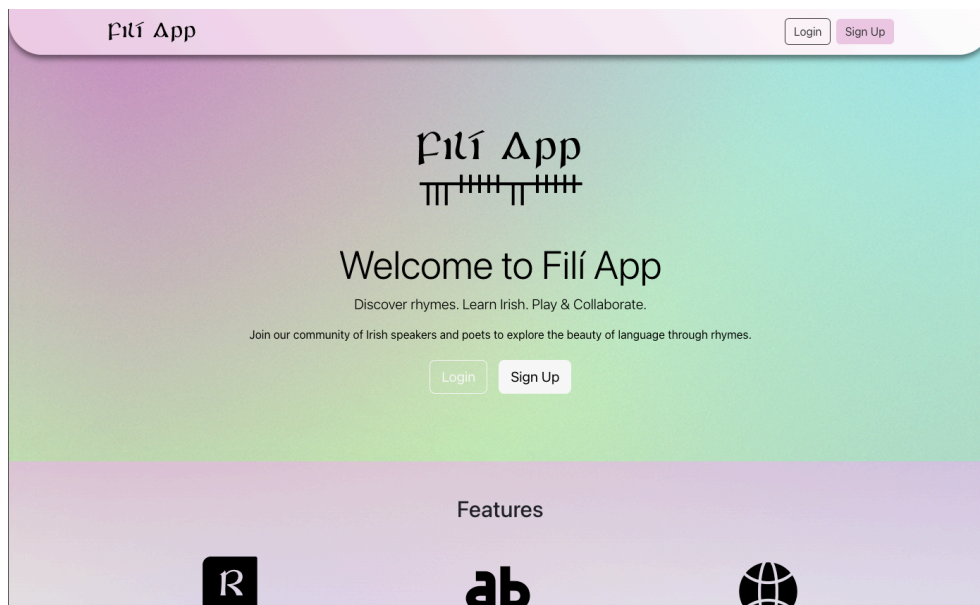


Figure 14. Landing Page GUI

Dashboard: Card-style entries with subtle CSS **hover** effects to signal interactivity and aid navigation.

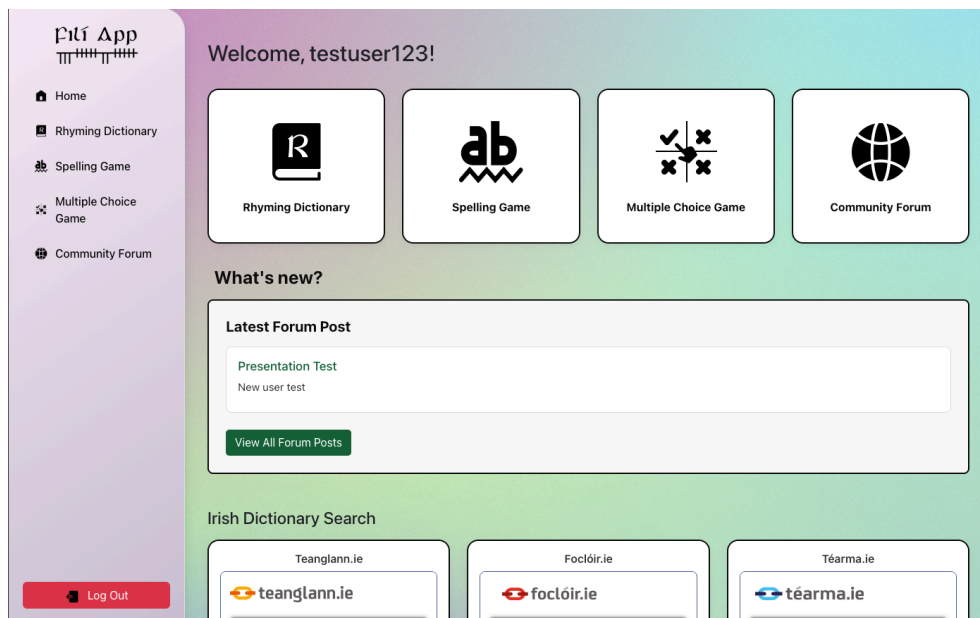


Figure 15. Dashboard Page GUI

Navigation: Persistent **side navbar** inside the app (common web-app pattern) for scalable, predictable routing.

Responsiveness: Layout **wraps** at Bootstrap breakpoints so the navbar and side panel don't overflow on small screens.

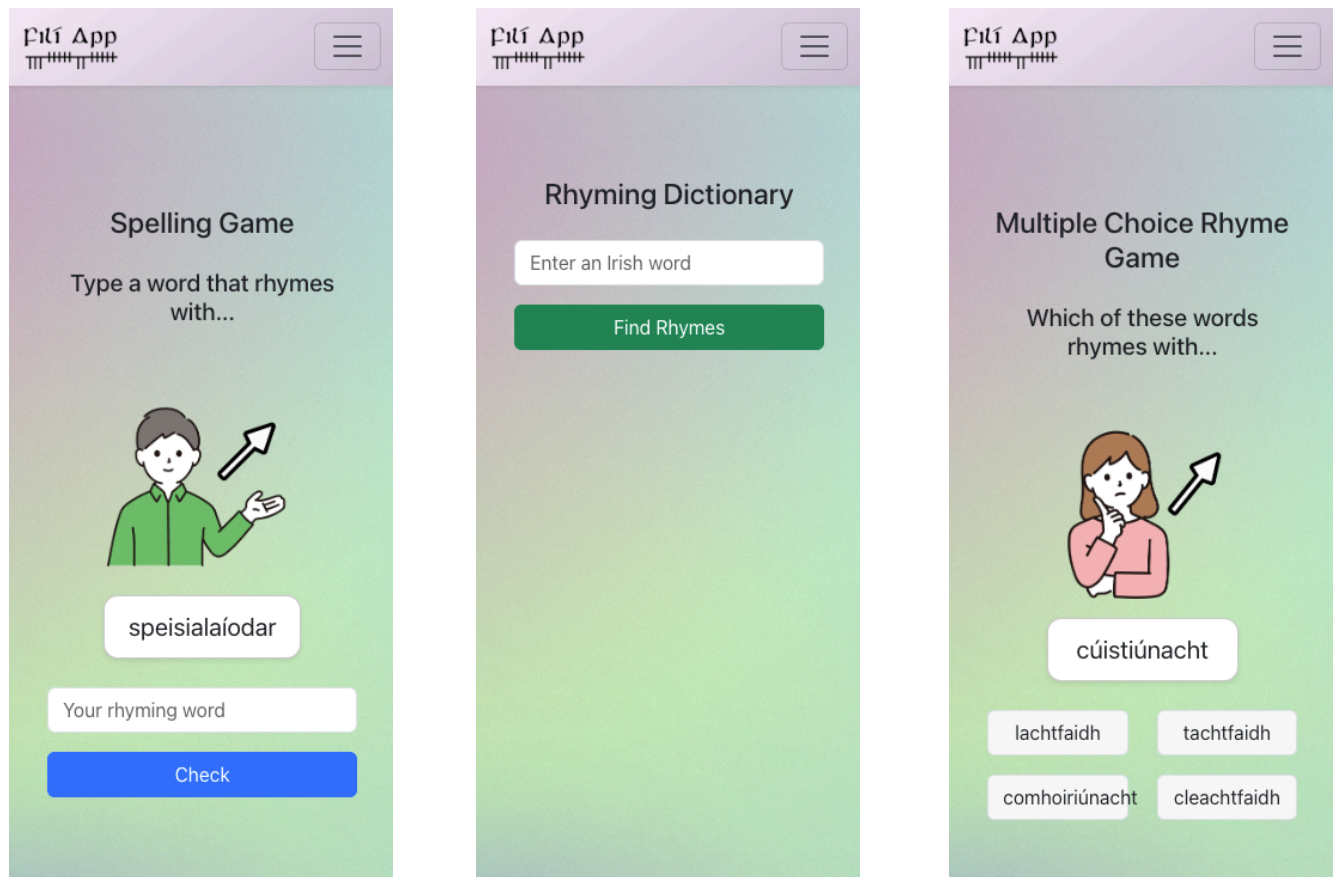


Figure 16. Mobile Screens GUI

3. Conclusions

The project delivers a working Irish rhyming dictionary with two games, session based authentication, and a simple forum. The cache first design makes lookups feel instant, and the clear rule set around Irish orthography keeps learning honest. The client, API, and database are cleanly separated, which helped the build stay understandable. Limits remain in search tolerance and game depth, and two planned features were not finished, a user form to request missing rhymes and an admin update flow. Overall the app meets its core goals, the core logic is sound, and it shows real potential for learners who want a modern, creative way to use Irish.

4. Further development or research

Next steps should focus on quality and learning value. Add unit and system tests, and tighten search by refining thresholds or adding smarter candidate filters. Curate a smaller specialist dataset and add columns for difficulty and for grouping by similar rhyme sounds so multiple choice rounds feel fair but challenging. Enrich prompts with images to trigger memory and audio to reinforce phonetics, including dialect recordings for advanced play. Finish the user suggestion workflow and the admin update tools. Longer term, study classroom use, track engagement and accuracy over time, and adapt difficulty based on performance to measure how a rhyme first approach supports Irish learning.

5. References

- bcryptjs. *npm*. Available at: <https://www.npmjs.com/package/bcryptjs>
- connect-flash. *npm*. Available at: <https://www.npmjs.com/package/connect-flash>
- cookie-parser. *npm*. Available at: <https://www.npmjs.com/package/cookie-parser>
- cors. *npm*. Available at: <https://www.npmjs.com/package/cors>
- dotenv. *npm*. Available at: <https://www.npmjs.com/package/dotenv>
- express. *npm*. Available at: <https://www.npmjs.com/package/express>
- express-session. *npm*. Available at: <https://www.npmjs.com/package/express-session>
- fuzzball. *npm*. Available at: <https://www.npmjs.com/package/fuzzball>
- mssql. *npm*. Available at: <https://www.npmjs.com/package/mssql>
- node-cache. *npm*. Available at: <https://www.npmjs.com/package/node-cache>
- passport. *npm*. Available at: <https://www.npmjs.com/package/passport>
- passport-local. *npm*. Available at: <https://www.npmjs.com/package/passport-local>
- axios. *npm*. Available at: <https://www.npmjs.com/package/axios>
- bootstrap. *npm*. Available at: <https://www.npmjs.com/package/bootstrap>
- react-jquery-plugin. *npm*. Available at: <https://www.npmjs.com/package/react-jquery-plugin>
- react-router-dom. *npm*. Available at: <https://www.npmjs.com/package/react-router-dom>
- ScienceDirect (n.d.) 'Phonology – Psychology'. *ScienceDirect Topics*. Available at: <https://www.sciencedirect.com/topics/psychology/phonology>
- Britannica (n.d.) 'Phoneme'. *Encyclopedia Britannica*. Available at: <https://www.britannica.com/topic/phoneme>
- Ó Scanaill, C. (2019) *Foclóir Rímeanna, 1.0*. Cadhan Aonair. Available at: <https://kevinscannell.com/files/foc.pdf>
- Kirkgöz, Y. (2014) 'Exploring poems to promote language learners' creative writing', *Procedia – Social and Behavioral Sciences*, Elsevier. Available at: <https://>

www.sciencedirect.com/science/article/pii/S1877042814061990?ref=pdf_download&fr=RR-2&rr=954b5923ae616d32

Luo, Z. (2023) 'The effectiveness of gamified tools for foreign language learning (FLL): A systematic review', *Behavioral Sciences*. Available at: <http://pmc.ncbi.nlm.nih.gov/articles/PMC10135444/#behavsci-13-00331-t004>

Grofčíková, S. and Máčajová, M. (2021) 'The impact of nursery rhymes on the development of phonological awareness in preschool children', *International Journal of Instruction*. Available at: <https://files.eric.ed.gov/fulltext/EJ1296910.pdf>

SVG Repo. (n.d.) *Icons and vector graphics*. Accessed via Figma.

Grainy Gradient. (n.d.) *Design asset*. Accessed via Figma.

Microsoft. (n.d.) *SQL Server*. Microsoft.

Microsoft. (n.d.) *Azure Data Studio*. Microsoft.

Docker. (n.d.) *Docker*. Docker Inc.

6. Appendix

6.1. Project Proposal

6.2. Project Plan

6.3. Requirement Specification